FieldOps Admin (Sistema de Gestión de Operaciones)

Contenido

FieldOps Admin (Sistema de Gestión de Operaciones) 1

1. Descripción del Proyecto 2
2. Objetivo del Proyecto 2
3. Alcance del Proyecto 2
4. Stack Tecnológico y Herramientas 3
5. Requisitos Funcionales 4
6. Requisitos No Funcionales 6
7. Historias de Usuario 7

1. Descripción del Proyecto

FieldOps Admin es una plataforma web B2B (Business-to-Business) diseñada para digitalizar, centralizar y automatizar la administración de empresas que prestan servicios técnicos o de campo.

El sistema resuelve el problema común de la desorganización operativa al eliminar el uso de hojas de cálculo manuales. Permite a los administradores gestionar su personal, controlar el inventario de equipos y asignar "Órdenes de Trabajo" de manera eficiente. Al mismo tiempo, proporciona a los técnicos de campo una interfaz móvil rápida para actualizar el estado de sus servicios en tiempo real. El enfoque principal del sistema es la trazabilidad, la seguridad de los datos y la eficiencia operativa.

2. Objetivo del Proyecto

- **Objetivo General:** Diseñar, desarrollar y desplegar un Sistema de Gestión de Operaciones (FieldOps Admin) end-to-end que centralice y automatice la asignación de servicios y el control de inventario. El propósito técnico de este proyecto es demostrar capacidad resolutiva full-stack, autonomía en la toma de decisiones arquitectónicas y la aplicación de metodologías de aseguramiento de calidad (QA) en un entorno de desarrollo web moderno.

- **Objetivos Específicos:**
  - Eliminar la dependencia de procesos manuales (hojas de cálculo) en la gestión de órdenes de trabajo.
  - Implementar un control de acceso seguro y basado en roles (RBAC).
  - Garantizar la estabilidad del sistema mediante la implementación de pruebas unitarias en la lógica crítica de negocio.
  - Construir una interfaz de usuario altamente responsiva y optimizada para uso en campo (dispositivos móviles) y administración en oficina (escritorio).

3. Alcance del Proyecto

Definir el alcance es crucial para evitar que el proyecto crezca sin control (el famoso *scope creep*). Esto le demuestra a un Tech Lead que sabes priorizar.

- **Dentro del Alcance (Lo que se va a construir):**
  - Sistema de autenticación y autorización (Login y Roles).
  - Módulo completo (CRUD) para la gestión de usuarios/técnicos.
  - Módulo completo (CRUD) para la gestión del inventario de equipos y sus estados.
  - Módulo transaccional para la creación, asignación y actualización de Órdenes de Trabajo.
  - Dashboard analítico con métricas básicas en tiempo real.
  - Pruebas unitarias para los endpoints críticos del backend (asignación de equipos y cambios de estado).

- **Fuera del Alcance (Lo que NO se va a construir en esta versión 1.0):**
  - **Para mantener el proyecto enfocado y terminarlo a tiempo, esta versión no incluirá***:* Pasarelas de pago o facturación electrónica, seguimiento por GPS en tiempo real de los técnicos, ni un sistema de chat interno integrado. Estas características podrán ser evaluadas para futuras iteraciones.

4. Stack Tecnológico y Herramientas

Esta sección detalla el ecosistema que soportará la aplicación, seleccionado por su alta demanda en el mercado laboral y su capacidad de escalabilidad.

- **Frontend (Interfaz de Usuario):**
  - **Librería Core:** React.
  - **Empaquetador:** Vite (para tiempos de carga y construcción ultrarrápidos).
  - **Estilos:** Tailwind CSS (diseño responsivo y utilitario).
  - **Lenguaje:** JavaScript / TypeScript (para mayor tipado y prevención de errores).

- **Backend (Lógica y API):**
  - **Entorno de Ejecución:** Node.js.
  - **Framework:** Express.js (para la creación de la API RESTful).
  - **Autenticación:** JSON Web Tokens (JWT) y Bcrypt (hashing de contraseñas).

- **Base de Datos y Almacenamiento:**
  - **Motor Relacional:** PostgreSQL o MySQL (para garantizar la integridad de las relaciones entre técnicos, equipos y órdenes).

- **Calidad, Pruebas y DevOps:**
  - **Testing:** Jest (para pruebas unitarias en la lógica del backend).
  - **Pruebas de API:** Postman (documentación y verificación de endpoints).
  - **Control de Versiones:** Git y GitHub.
  - **Auditoría de Código:** Uso de herramientas de Inteligencia Artificial para el análisis estático del código, revisión de vulnerabilidades y optimización de consultas a la base de datos.

5. Requisitos Funcionales

Estos definen lo que el sistema **debe hacer** (sus comportamientos y funciones).

- **Módulo de Autenticación y Usuarios**
  - **RF-01 | Autenticación Segura:** El sistema debe permitir el inicio de sesión mediante correo electrónico y contraseña encriptada.
  - **RF-02 | Control de Acceso Basado en Roles (RBAC):** El sistema debe manejar al menos dos roles con permisos distintos:
    - ✓ ***Administrador**:* Acceso total al sistema (crear, leer, actualizar y eliminar cualquier registro).
    - ✓ ***Técnico**:* Acceso restringido; solo puede ver las órdenes de trabajo que se le han asignado y actualizar su estado.

- **RF-03 | Gestión de Personal:** El administrador debe poder registrar nuevos técnicos, editar su información de contacto y desactivar sus cuentas (Soft Delete).

❖ **Módulo de Inventario de Equipos**

- **RF-04 | CRUD de Activos:** El sistema debe permitir registrar, visualizar, editar y dar de baja equipos o herramientas de la empresa.
- **RF-05 | Trazabilidad de Estados:** Cada equipo debe tener un estado dinámico (Ej: "Disponible", "En Mantenimiento", "Asignado").
- **RF-06 | Búsqueda y Filtrado:** El usuario debe poder buscar equipos por número de serie o filtrar el listado por su estado actual.

❖ **Módulo de Órdenes de Trabajo (Core del Negocio)**

- **RF-07 | Creación de Órdenes:** El administrador debe poder crear una orden de trabajo especificando: título, descripción del problema, fecha programada, cliente, técnico asignado y equipos requeridos.
- **RF-08 | Prevención de Conflictos:** El sistema no debe permitir asignar a una orden de trabajo un equipo que tenga el estado "En Mantenimiento" o "Asignado" a otra orden en curso.
- **RF-09 | Actualización de Flujo:** El técnico asignado debe poder cambiar el estado de la orden (Ej: "Pendiente" -> "En Progreso" -> "Completada").
- **RF-10 | Historial de Cambios:** El sistema debe registrar qué usuario cambió el estado de una orden y la fecha/hora exacta en que lo hizo.

❖ **Módulo de Dashboard y Reportes**

- **RF-11 | Métricas Generales:** La pantalla de inicio del administrador debe mostrar tarjetas de resumen (Ej: Total de órdenes completadas en el mes, cantidad de técnicos activos, equipos en mantenimiento).

6. Requisitos No Funcionales

Estos definen **cómo** el sistema debe comportarse (rendimiento, seguridad, stack tecnológico y estándares de calidad).

❖ Stack Tecnológico y Arquitectura

- **RNF-01 | Frontend:** La interfaz de usuario debe estar desarrollada como una Single Page Application (SPA) utilizando **React** (con Vite para la construcción) y **Tailwind CSS** para los estilos.
- **RNF-02 | Backend:** La lógica de servidor y la API RESTful deben estar construidas con **Node.js y Express**.
- **RNF-03 | Base de Datos:** El almacenamiento de datos debe realizarse en un motor relacional como **PostgreSQL o MySQL**, garantizando la integridad referencial (ej. que no se borre un técnico si tiene órdenes asignadas).

❖ Calidad de Software y Pruebas

- **RNF-04 | Cobertura de Pruebas Unitarias:** Los endpoints críticos del backend (como la creación de órdenes de trabajo y la asignación de inventario) deben contar con pruebas automatizadas que validen tanto los escenarios de éxito como los de error.
- **RNF-05 | Auditoría de Código:** La arquitectura y las consultas a la base de datos deben estar auditadas para garantizar eficiencia y prevenir cuellos de botella.

❖ Seguridad

- **RNF-06 | Manejo de Sesiones:** La autorización de las rutas del backend debe realizarse mediante la validación de tokens **JWT (JSON Web Tokens)** enviados en las cabeceras HTTP.
- **RNF-07 | Protección de Datos:** Las contraseñas de los usuarios deben ser hasheadas (ej. con Bcrypt) antes de ser almacenadas en la base de datos.
- **RNF-08 | Sanitización de Entradas:** Todos los datos enviados desde el frontend (formularios) deben ser validados y sanitizados en el backend para prevenir ataques de inyección SQL y XSS.

❖ **Experiencia de Usuario (UX) e Interfaz (UI)**

- **RNF-09 | Diseño Responsivo (Mobile-First):** Las vistas del Técnico (listado de órdenes y actualización de estado) deben ser 100% funcionales y legibles en dispositivos móviles, ya que se usarán en trabajo de campo.
- **RNF-10 | Retroalimentación del Sistema:** Todas las acciones de creación, edición o error deben mostrar notificaciones claras (Toasts o Modales) al usuario.

7. Historias de Usuario

ÉPICA 1: Autenticación y Seguridad

HU 1.1 - Inicio de Sesión Seguro

- **Historia:** Como usuario (Administrador o Técnico), quiero iniciar sesión con mi correo electrónico y contraseña para acceder a mi panel correspondiente de forma segura.
- **Criterios de Aceptación:**
    1. Si las credenciales son correctas, el backend (Express) debe generar un token JWT y el frontend (React) debe redirigir al Dashboard.
    2. Si la contraseña es incorrecta o el usuario no existe, se debe mostrar un mensaje de error claro ("Credenciales inválidas") sin revelar si el error fue el correo o la clave.
    3. Las contraseñas en la base de datos deben estar encriptadas (ej. usando Bcrypt).

HU 1.2 - Control de Accesos (RBAC)

- **Historia:** Como sistema, quiero restringir la navegación y las acciones según el rol del usuario autenticado para evitar filtraciones de datos o acciones no autorizadas.
- **Criterios de Aceptación:**
    1. Un Técnico que intente acceder a la URL /admin/usuarios debe ser redirigido a una vista de "Acceso Denegado".

2. Los endpoints protegidos en la API deben tener un middleware que verifique la validez del JWT y el rol del usuario antes de ejecutar la consulta a la base de datos.
3. El menú de navegación en el frontend debe ocultar los botones de administración si el usuario es un Técnico.

ÉPICA 2: Gestión del Personal y Usuarios

HU 2.1 - Creación de Técnicos

- **Historia:** Como Administrador, quiero poder registrar a un nuevo técnico en el sistema para poder asignarle órdenes de trabajo.
- **Criterios de Aceptación:**
    1. El formulario debe validar que el correo electrónico tenga un formato correcto y no esté registrado previamente.
    2. Al crear el usuario, el sistema le asigna automáticamente el rol de "Técnico" y un estado "Activo".

HU 2.2 - Desactivación de Usuarios

- **Historia:** Como Administrador, quiero desactivar la cuenta de un técnico en lugar de borrarla, para mantener el historial de las órdenes que completó en el pasado.
- **Criterios de Aceptación:**
    1. La base de datos debe utilizar un "Soft Delete" (cambiar un campo booleano activo a false en lugar de ejecutar un DELETE).
    2. Un usuario inactivo no debe poder iniciar sesión en el sistema.

ÉPICA 3: Inventario de Activos

HU 3.1 - Registro y Listado de Equipos

- **Historia:** Como Administrador, quiero registrar nuevos equipos y ver una lista completa del inventario para conocer con qué herramientas cuenta la empresa.
- **Criterios de Aceptación:**
    1. La tabla debe mostrar: Nombre, Número de Serie, y Estado actual (Disponible, Asignado, Mantenimiento).
    2. Debe existir una barra de búsqueda que filtre los equipos por número de serie en tiempo real.

HU 3.2 - Cambio de Estado por Mantenimiento

- **Historia:** Como Administrador, quiero cambiar el estado de un equipo a "Mantenimiento" para evitar que sea asignado a una nueva orden de trabajo si está averiado.
- **Criterios de Aceptación:**
    1. El sistema no debe permitir pasar a "Mantenimiento" un equipo que actualmente tenga el estado "Asignado" a una orden en progreso (debe liberar primero la orden).

ÉPICA 4: Core de Negocio - Órdenes de Trabajo (FieldOps)

HU 4.1 - Asignación de Servicios

- **Historia:** Como Administrador, quiero crear una orden de trabajo seleccionando un cliente, un técnico y un equipo necesario, para despachar el servicio.
- **Criterios de Aceptación:**
    1. El formulario (en el frontend) solo debe listar en los menús desplegables a los técnicos con estado "Activo" y a los equipos con estado "Disponible".

2. **(Punto crítico para prueba unitaria):** Si mediante Postman o la API se intenta forzar la asignación de un equipo "En Mantenimiento", el backend debe rechazar la solicitud con un código HTTP 409 (Conflict).
3. Al crearse la orden, el estado del equipo seleccionado debe cambiar automáticamente a "Asignado".

HU 4.2 - Vista Móvil del Técnico

- **Historia:** Como Técnico en campo, quiero ver una lista optimizada para celulares con las órdenes que tengo asignadas hoy para saber adónde debo ir.
- **Criterios de Aceptación:**
    1. La vista debe ser 100% responsiva (usando clases de TailwindCSS como flex-col en pantallas pequeñas).
    2. El técnico solo debe ver las órdenes donde su ID coincide con el id_tecnico de la orden. Nunca debe ver las de sus compañeros.

HU 4.3 - Actualización de Progreso en Tiempo Real

- **Historia:** Como Técnico, quiero poder actualizar el estado de mi orden (Pendiente -> En Progreso -> Completada) para informar a la administración que terminé el trabajo.
- **Criterios de Aceptación:**
    1. El técnico debe tener un botón claro para cambiar el estado.
    2. **(Punto crítico de arquitectura):** Cuando una orden pasa al estado "Completada", el sistema debe buscar el equipo asociado a esa orden y cambiar automáticamente su estado de "Asignado" a "Disponible" en el inventario.

ÉPICA 5: Dashboard y Analítica

HU 5.1 - Tarjetas de Resumen (KPIs)

- **Historia:** Como Administrador, quiero ver tarjetas de resumen al iniciar sesión para conocer la salud operativa de la empresa de un vistazo.
- **Criterios de Aceptación:**
    1. El panel debe mostrar al menos tres métricas: "Órdenes Completadas (Mes actual)", "Técnicos Activos", y "Equipos en Mantenimiento".
    2. Los datos deben ser calculados dinámicamente por el backend mediante consultas eficientes a la base de datos, no iterando arrays gigantes en el frontend.

8. Conclusión

El desarrollo de **FieldOps Admin** representa la creación de una solución tecnológica integral orientada a resolver problemas reales de desorganización operativa, pérdida de información y cuellos de botella en el sector de servicios. Al reemplazar procesos manuales con un sistema centralizado, se optimiza la comunicación entre la administración y el personal de campo, garantizando la trazabilidad de cada activo y orden de trabajo.

Más allá de su utilidad comercial, la arquitectura de este proyecto está diseñada bajo estrictos estándares de la industria. La integración de un stack moderno, sumada a la implementación de pruebas automatizadas, control de roles y auditoría continua, asegura que el sistema sea escalable, seguro y fácil de mantener.

La ejecución de FieldOps Admin evidencia la capacidad para gestionar el ciclo de vida completo del desarrollo de software: desde el levantamiento de requerimientos y el diseño relacional de la base de datos, hasta la construcción de interfaces responsivas y el despliegue de una API robusta, cumpliendo a cabalidad con las exigencias de un entorno tecnológico profesional.